

2º curso / 2º cuatr.

Grado en
Ingeniería
Informática

Arquitectura de Computadores (AC)

1.1.1 Cuaderno de prácticas.

1.1.2 Bloque Práctico 3. Programación paralela III: Interacción con el entorno en OpenMP

Estudiante (nombre y apellidos): David Rodriguez Aparicio

Parte I. Ejercicios basados en los ejemplos del seminario práctico

1. Usar la cláusula `num_threads(x)` en el ejemplo del seminario `if_clause.c`, y añadir un parámetro de entrada al programa que fije el valor `x` que se va a usar en la cláusula. Incorporar en el cuaderno de trabajo de esta práctica volcados de pantalla con ejemplos de ejecución que ilustren la funcionalidad de esta cláusula y explicar por qué lo ilustran.

CAPTURA CÓDIGO FUENTE: if-clauseModificado.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(int argc, char **argv)
{
    int i, n = 20, tid;
    int a[20], suma = 0, sumalocal;
    int x;

    if (argc < 3) {
        fprintf(stderr, "[ERROR] Uso: %s <iteraciones> <num_threads>\n",
argv[0]);
        exit(-1);
    }

    n = atoi(argv[1]);
    if (n > 20) n = 20;

    x = atoi(argv[2]);

    for (i = 0; i < n; i++) {
        a[i] = i;
    }

    #pragma omp parallel if(n > 4) num_threads(x) default(none) \
        private(sumalocal, tid, i) shared(a, suma, n)
    {
        sumalocal = 0;
```

```

    tid = omp_get_thread_num();

#pragma omp for schedule(static) nowait
for (i = 0; i < n; i++) {
    sumalocal += a[i];
    printf("thread %d suma de a[%d]=%d sumalocal=%d\n",
           tid, i, a[i], sumalocal);
}

#pragma omp atomic
suma += sumalocal;

#pragma omp barrier

#pragma omp master
printf("thread master=%d imprime suma=%d\n", tid, suma);
}

return 0;
}

```

CAPTURAS DE PANTALLA:

```

PS C:\Users\Daviton\Desktop\Ingenieria-Informatica-2\AC\Practica 3 (+ S3)> gcc -O2 -fopenmp -o if-clauseModificado if-clauseModificado.c
PS C:\Users\Daviton\Desktop\Ingenieria-Informatica-2\AC\Practica 3 (+ S3)> ./if-clauseModificado 4 3
thread 0 suma de a[0]=0 sumalocal=0
thread 0 suma de a[1]=1 sumalocal=1
thread 0 suma de a[2]=2 sumalocal=3
thread 0 suma de a[3]=3 sumalocal=6
thread master=0 imprime suma=6
PS C:\Users\Daviton\Desktop\Ingenieria-Informatica-2\AC\Practica 3 (+ S3)> ./if-clauseModificado 5 3
thread 0 suma de a[0]=0 sumalocal=0
thread 0 suma de a[1]=1 sumalocal=1
thread 1 suma de a[2]=2 sumalocal=2
thread 1 suma de a[3]=3 sumalocal=5
thread 2 suma de a[4]=4 sumalocal=4
thread master=0 imprime suma=10
PS C:\Users\Daviton\Desktop\Ingenieria-Informatica-2\AC\Practica 3 (+ S3)> ./if-clauseModificado 8 4
thread 0 suma de a[0]=0 sumalocal=0
thread 0 suma de a[1]=1 sumalocal=1
thread 1 suma de a[2]=2 sumalocal=2
thread 1 suma de a[3]=3 sumalocal=5
thread 3 suma de a[6]=6 sumalocal=6
thread 3 suma de a[7]=7 sumalocal=13
thread 2 suma de a[4]=4 sumalocal=4
thread 2 suma de a[5]=5 sumalocal=9
thread master=0 imprime suma=28
PS C:\Users\Daviton\Desktop\Ingenieria-Informatica-2\AC\Practica 3 (+ S3)>

```

RESPUESTA:

Se ha añadido num_threads para trabajar con hilos, si $n > 4$ se activa la region paralela y si $n \leq 4$ se usa solo el hilo 0.

2. Rellenar la Tabla 1 (se debe poner en la tabla el id del *thread* que ejecuta cada iteración) usando scheduler-clause.c (este código está en el seminario) con tres *threads* (0,1,2) y un número de iteraciones de 16 (0 a 15 en la tabla). Con este ejercicio se pretende comparar distintas alternativas de planificación de bucles. Se van a usar distintos tipos (static, dynamic, guided), modificadores (monotonic y nonmonotonic) y tamaños de chunk ($x = 1$ y 2).

Tabla 1 . Tabla schedule. Rellenar esta tabla ejecutando scheduler-clause.c asignando previamente a la variable de entorno OMP_SCHEDULE los valores que se indican en la tabla (por ej.: export OMP_SCHEDULE="nonmonotonic:static,2"). En la segunda fila, 1 y 2 representan el tamaño del chunk

Iteración	"monotonic:static,x"		"nonmonotonic:static,x"		"monotonic:dynamic,x"		"monotonic:guided,x"	
	x=1	x=2	x=1	x=2	x=1	x=2	x=1	x=2
0	0	0	0	0	0	1	0	0
1	1	0	1	0	1	1	0	0
2	2	1	2	1	0	0	0	0
3	0	1	0	1	1	0	0	0
4	1	2	1	2	1	2	0	0
5	2	2	2	2	1	2	0	0
6	0	0	0	0	1	2	1	1
7	1	0	1	0	1	2	1	1
8	2	1	2	1	1	0	1	1
9	0	1	0	1	1	0	1	1
10	1	2	1	2	1	0	2	2
11	2	2	2	2	1	0	2	2
12	0	0	0	0	1	0	1	1
13	1	0	1	0	1	0	1	1
14	2	1	2	1	1	0	1	1
15	0	1	0	1	1	0	1	1

Destacar las diferencias entre las 4 alternativas de planificación de la tabla, en particular, las que hay entre static, dynamic y guided y las diferencias entre usar monotonic y nonmonotonic.

RESPUESTA:

En static se reparten de forma fija y predecible, en dynamic los hilos van cogiendo varias iteraciones y puede cambiar, en guided se reparten en conjuntos grandes de interacciones y luego se reduce.

Monotonic garantiza mas orden en las asignaciones, y nonmonotonic es algo mas impredecible.

3. ¿Qué valor por defecto usa OpenMP para chunk y modifier con static, dynamic y guided? Explicar qué has hecho para contestar a esta pregunta.

He ejecutado scheduler-clause con runtime y cambiando el schedule entre static-dynamic-guided. static reparte bloques entre los hilos, y dynamic y guided usan planificación en tiempo de ejecucion.

El modificador por defecto suele ser monotonic solo en static, para los otros depende de la ejecución, ya que puede variar.

4. Añadir al programa scheduled-clause.c lo necesario para que imprima el valor de las variables de control dyn-var, nthreads-var, thread-limit-var y run-sched-var dentro (debe imprimir sólo un thread) y fuera de la región paralela. Realizar varias ejecuciones usando variables de entorno para modificar estas variables de control antes de la ejecución. Incorporar en el cuaderno de prácticas volcados de pantalla de estas ejecuciones. ¿Se imprimen valores distintos dentro y fuera de la región paralela?

CAPTURA CÓDIGO FUENTE: scheduled-clauseModificado.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void imprimir_variaciones(char *zona)
{
    omp_sched_t kind;
    int chunk;

    omp_get_schedule(&kind, &chunk);

    printf("\n--- %s ---\n", zona);
    printf("dyn-var = %d\n", omp_get_dynamic());
    printf("nthreads-var = %d\n", omp_get_max_threads());
    printf("thread-limit-var = %d\n", omp_get_thread_limit());
    printf("run-sched-var: kind = %d, chunk = %d\n", kind, chunk);
}

int main(int argc, char **argv)
{
    int i, n = 200, chunk, a[200], suma = 0;

    if (argc < 3) {
        fprintf(stderr, "\nFalta iteraciones o chunk\n");
        exit(-1);
    }

    n = atoi(argv[1]);
    if (n > 200) n = 200;

    chunk = atoi(argv[2]);

    for (i = 0; i < n; i++)
        a[i] = i;
```

```

    imprimir_variables("Fuera de la region paralela");

#pragma omp parallel
{
    #pragma omp single
    imprimir_variables("Dentro de la region paralela");

    #pragma omp for firstprivate(suma) lastprivate(suma)
schedule(dynamic, chunk)
    for (i = 0; i < n; i++) {
        suma = suma + a[i];
        printf("thread %d suma a[%d]=%d suma=%d\n",
            omp_get_thread_num(), i, a[i], suma);
    }
}

printf("Fuera de 'parallel for' suma=%d\n", suma);

return 0;
}

```

CAPTURAS DE PANTALLA:

```

david@DESKTOP-BQGQKB2:/mnt/c/Users/Davito/Desktop/Ingenieria-Informatica-2/AC/Practica 3 (+ S3)$ gcc -O2 -fopenmp -o schedul
david@DESKTOP-BQGQKB2:/mnt/c/Users/Davito/Desktop/Ingenieria-Informatica-2/AC/Practica 3 (+ S3)$ export OMP_NUM_THREADS=3
export OMP_DYNAMIC=FALSE
export OMP_SCHEDULE="static,1"
./scheduled-clauseModificado 16 2

--- Fuera de la region paralela ---
dyn-var = 0
nthreads-var = 3
thread-limit-var = 2147483647
run-sched-var: kind = -2147483647, chunk = 1

--- Dentro de la region paralela ---
dyn-var = 0
nthreads-var = 3
thread-limit-var = 2147483647
run-sched-var: kind = -2147483647, chunk = 1
thread 0 suma a[4]=4 suma=4
thread 2 suma a[2]=2 suma=2
thread 2 suma a[3]=3 suma=5
thread 2 suma a[6]=6 suma=11
thread 2 suma a[7]=7 suma=18
thread 2 suma a[8]=8 suma=26
thread 2 suma a[9]=9 suma=35
thread 2 suma a[10]=10 suma=45
thread 2 suma a[11]=11 suma=56
thread 2 suma a[12]=12 suma=68
thread 2 suma a[13]=13 suma=81
thread 0 suma a[5]=5 suma=9
thread 1 suma a[0]=0 suma=0
thread 1 suma a[1]=1 suma=1
thread 2 suma a[14]=14 suma=95
thread 2 suma a[15]=15 suma=110
Fuera de 'parallel for' suma=110

```

```

david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 3 (+ S3)$ export OMP_NUM_THREADS=4
export OMP_DYNAMIC=TRUE
export OMP_SCHEDULE="dynamic,2"
./scheduled-clauseModificado 16 2

--- Fuera de la region paralela ---
dyn-var = 1
nthreads-var = 4
thread-limit-var = 2147483647
run-sched-var: kind = 2, chunk = 2

--- Dentro de la region paralela ---
dyn-var = 1
nthreads-var = 4
thread-limit-var = 2147483647
run-sched-var: kind = 2, chunk = 2
thread 0 suma a[6]=6 suma=6
thread 0 suma a[7]=7 suma=13
thread 0 suma a[8]=8 suma=21
thread 0 suma a[9]=9 suma=30
thread 0 suma a[10]=10 suma=40
thread 0 suma a[11]=11 suma=51
thread 0 suma a[12]=12 suma=63
thread 0 suma a[13]=13 suma=76
thread 0 suma a[14]=14 suma=90
thread 0 suma a[15]=15 suma=105
thread 2 suma a[2]=2 suma=2
thread 2 suma a[3]=3 suma=5
thread 3 suma a[0]=0 suma=0
thread 3 suma a[1]=1 suma=1
thread 1 suma a[4]=4 suma=4
thread 1 suma a[5]=5 suma=9
Fuera de 'parallel for' suma=105

```

RESPUESTA:

Se han usado las funciones de OpenMP para las variables de control, los valores suelen ser iguales dentro y fuera, ya que las variables de entorno se fijan antes de ejecutar el programa.

5. Usar en el ejemplo anterior las funciones `omp_get_num_threads()`, `omp_get_num_procs()` y `omp_in_parallel()` dentro y fuera de la región paralela. Imprimir los valores que obtienen estas funciones dentro (lo debe imprimir sólo uno de los threads) y fuera de la región paralela. Incorporar en el cuaderno de prácticas volcados de pantalla con los resultados de ejecución obtenidos. Indicar en qué funciones se obtienen valores distintos dentro y fuera de la región paralela.

CAPTURA CÓDIGO FUENTE: scheduled-clauseModificado2.c

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void imprimir_funciones(char *zona)
{
    printf("\n--- %s ---\n", zona);
    printf("omp_get_num_threads() = %d\n", omp_get_num_threads());
    printf("omp_get_num_procs() = %d\n", omp_get_num_procs());
    printf("omp_in_parallel() = %d\n", omp_in_parallel());
}

```

```

int main(int argc, char **argv)
{
    int i, n = 200, chunk, a[200], suma = 0;

    if (argc < 3) {
        fprintf(stderr, "\nFalta iteraciones o chunk\n");
        exit(-1);
    }

    n = atoi(argv[1]);
    if (n > 200) n = 200;

    chunk = atoi(argv[2]);

    for (i = 0; i < n; i++)
        a[i] = i;

    imprimir_funciones("Fuera de la region paralela");

    #pragma omp parallel
    {
        #pragma omp single
        imprimir_funciones("Dentro de la region paralela");

        #pragma omp for firstprivate(suma) lastprivate(suma)
        schedule(dynamic, chunk)
        for (i = 0; i < n; i++) {
            suma = suma + a[i];
            printf("thread %d suma a[%d]=%d suma=%d\n",
                omp_get_thread_num(), i, a[i], suma);
        }
    }

    printf("Fuera de 'parallel for' suma=%d\n", suma);

    return 0;
}

```

CAPTURAS DE PANTALLA:

```
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 3 (+ S3)$ export OMP_NUM_THREADS=3
./scheduled-clauseModificado2 16 2

--- Fuera de la region paralela ---
omp_get_num_threads() = 1
omp_get_num_procs() = 6
omp_in_parallel() = 0

--- Dentro de la region paralela ---
omp_get_num_threads() = 3
omp_get_num_procs() = 6
omp_in_parallel() = 1
thread 0 suma a[4]=4 suma=4
thread 0 suma a[5]=5 suma=9
thread 0 suma a[6]=6 suma=15
thread 0 suma a[7]=7 suma=22
thread 0 suma a[8]=8 suma=30
thread 0 suma a[9]=9 suma=39
thread 0 suma a[10]=10 suma=49
thread 0 suma a[11]=11 suma=60
thread 0 suma a[12]=12 suma=72
thread 0 suma a[13]=13 suma=85
thread 0 suma a[14]=14 suma=99
thread 0 suma a[15]=15 suma=114
thread 1 suma a[0]=0 suma=0
thread 1 suma a[1]=1 suma=1
thread 2 suma a[2]=2 suma=2
thread 2 suma a[3]=3 suma=5
Fuera de 'parallel for' suma=114
```

RESPUESTA:

omp_get_num_threads() cambia porque fuera de la region paralela vale 1, dentro devuelve el numero de threads usados.

omp_in_parallel() tambien cambia ya que fuera vale 0 y dentro 1, digamos que comprueba que esta dentro de la region paralela.

Y get_num_process() no cambia porque simplemente indica el numero de procesadores disponibles.

6. Añadir al programa scheduled-clause.c lo necesario para, usando funciones, modificar las variables de control dyn-var, nthreads-var y run-sched-var dentro de la región paralela y fuera de la región paralela. En la modificación de run-sched-var se debe usar un valor de kind distinto al utilizado en la cláusula schedule(). Añadir lo necesario para imprimir el contenido de estas variables antes y después de cada una de las dos modificaciones. Comentar los resultados.

CAPTURA CÓDIGO FUENTE: scheduled-clauseModificado3.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void imprimir_variaciones(char *zona)
{
    omp_sched_t kind;
    int chunk;
```

```

    omp_get_schedule(&kind, &chunk);

    printf("\n--- %s ---\n", zona);
    printf("dyn-var = %d\n", omp_get_dynamic());
    printf("nthreads-var = %d\n", omp_get_max_threads());
    printf("run-sched-var: kind = %d, chunk = %d\n", kind, chunk);
}

int main(int argc, char **argv)
{
    int i, n = 200, chunk, a[200], suma = 0;

    if (argc < 3) {
        fprintf(stderr, "\nFalta iteraciones o chunk\n");
        exit(-1);
    }

    n = atoi(argv[1]);
    if (n > 200) n = 200;

    chunk = atoi(argv[2]);

    for (i = 0; i < n; i++)
        a[i] = i;

    imprimir_variables("Antes de modificar fuera");

    omp_set_dynamic(0);
    omp_set_num_threads(4);
    omp_set_schedule(omp_sched_static, 2);

    imprimir_variables("Despues de modificar fuera");

    #pragma omp parallel
    {
        #pragma omp single
        {
            imprimir_variables("Antes de modificar dentro");

            omp_set_dynamic(1);
            omp_set_num_threads(2);
            omp_set_schedule(omp_sched_guided, 1);

```

```
        imprimir_variables("Despues de modificar dentro");
    }

    #pragma omp for firstprivate(suma) lastprivate(suma)
    schedule(dynamic, chunk)
    for (i = 0; i < n; i++) {
        suma = suma + a[i];
        printf("thread %d suma a[%d]=%d suma=%d\n",
            omp_get_thread_num(), i, a[i], suma);
    }
}

printf("Fuera de 'parallel for' suma=%d\n", suma);

return 0;
}
```

CAPTURAS DE PANTALLA:

```
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 3 (+ S3)$ gcc -O2 -fopenmp -o scheduled-clauseModificado3 scheduled-clauseModificado3.c
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 3 (+ S3)$ ./scheduled-clauseModificado3 16 2

--- Antes de modificar fuera ---
dyn-var = 1
nthreads-var = 3
run-sched-var: kind = 2, chunk = 2

--- Despues de modificar fuera ---
dyn-var = 0
nthreads-var = 4
run-sched-var: kind = 1, chunk = 2

--- Antes de modificar dentro ---
dyn-var = 0
nthreads-var = 4
run-sched-var: kind = 1, chunk = 2

--- Despues de modificar dentro ---
dyn-var = 1
nthreads-var = 2
run-sched-var: kind = 3, chunk = 1
thread 0 suma a[4]=4 suma=4
thread 3 suma a[6]=6 suma=6
thread 3 suma a[7]=7 suma=13
thread 3 suma a[8]=8 suma=21
thread 3 suma a[9]=9 suma=30
thread 3 suma a[10]=10 suma=40
thread 3 suma a[11]=11 suma=51
thread 3 suma a[12]=12 suma=63
thread 3 suma a[13]=13 suma=76
thread 3 suma a[14]=14 suma=90
thread 3 suma a[15]=15 suma=105
thread 2 suma a[0]=0 suma=0
thread 2 suma a[1]=1 suma=1
thread 1 suma a[2]=2 suma=2
thread 1 suma a[3]=3 suma=5
thread 0 suma a[5]=5 suma=9
Fuera de 'parallel for' suma=105
```

RESPUESTA: Fuera de la region paralela, las funciones modifican las variables de control, dentro tambien se pueden cambiar pero no afectan a la region paralela que ya está en ejecución.

Aunque se modifique run-sched-var, el bucle usa la planificación indicada anteriormente, porque la clausula tiene prioridad.

Parte II. Resto de ejercicios (trabajar en local o en ATCGrid)

7. Implementar en paralelo con OpenMP la multiplicación de una matriz triangular superior por un vector a partir del código disponible en PRADO. El código debe repartir entre los threads las iteraciones del bucle que recorre las filas. Añadir lo necesario para que el usuario pueda fijar la planificación de tareas usando la variable de entorno OMP_SCHEDULE. Mostrar en una captura de pantalla que el código resultante funciona correctamente.

CAPTURA CÓDIGO FUENTE: pmtv-OpenMP.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(int argc, char **argv)
{
    int N, i, j;
    double **M, *v, *res;
    double t;

    if (argc < 2) {
```

```

    printf("Uso: %s <tamano_matriz>\n", argv[0]);
    return -1;
}

N = atoi(argv[1]);

M = malloc(N * sizeof(double *));
v = malloc(N * sizeof(double));
res = malloc(N * sizeof(double));

for (i = 0; i < N; i++) {
    M[i] = malloc(N * sizeof(double));
}

for (i = 0; i < N; i++) {
    v[i] = 1.0;
    res[i] = 0.0;
    for (j = 0; j < N; j++) {
        if (j >= i)
            M[i][j] = 1.0;
        else
            M[i][j] = 0.0;
    }
}

t = omp_get_wtime();

#pragma omp parallel for private(j) schedule(runtime)
for (i = 0; i < N; i++) {
    for (j = i; j < N; j++) {
        res[i] += M[i][j] * v[j];
    }
}

t = omp_get_wtime() - t;

printf("Tiempo: %f segundos\n", t);

if (N <= 20) {
    printf("Resultado:\n");
    for (i = 0; i < N; i++) {
        printf("res[%d] = %.2f\n", i, res[i]);
    }
}

```

```

    }

    for (i = 0; i < N; i++) {
        free(M[i]);
    }

    free(M);
    free(v);
    free(res);

    return 0;
}

```

DESCOMPOSICIÓN DE DOMINIO: Se paraleliza el bucle exterior que recorre las filas de la matriz triangular superior.

Cada thread calcula distintas filas del vector, indicado por OMP_SCHEDULE, y como las primeras filas tienen más elementos que las últimas no todas las filas tienen el mismo coste de computación.

CAPTURAS DE PANTALLA:

```

david@DESKTOP-BQGQKB2:/mnt/c/Users/Davito/Desktop/Ingenieria-Informatica-2/AC/Practica 3 (+ S3)$ gcc -O2 -fopenmp -o pmtv-OpenMP pmtv-OpenMP.c
david@DESKTOP-BQGQKB2:/mnt/c/Users/Davito/Desktop/Ingenieria-Informatica-2/AC/Practica 3 (+ S3)$ export OMP_NUM_THREADS=3
export OMP_SCHEDULE="static,1"
./pmtv-OpenMP 8
Tiempo: 0.000269 segundos
Resultado:
res[0] = 8.00
res[1] = 7.00
res[2] = 6.00
res[3] = 5.00
res[4] = 4.00
res[5] = 3.00
res[6] = 2.00
res[7] = 1.00
david@DESKTOP-BQGQKB2:/mnt/c/Users/Davito/Desktop/Ingenieria-Informatica-2/AC/Practica 3 (+ S3)$ export OMP_SCHEDULE="dynamic,1"
./pmtv-OpenMP 8
Tiempo: 0.000246 segundos
Resultado:
res[0] = 8.00
res[1] = 7.00
res[2] = 6.00
res[3] = 5.00
res[4] = 4.00
res[5] = 3.00
res[6] = 2.00
res[7] = 1.00
david@DESKTOP-BQGQKB2:/mnt/c/Users/Davito/Desktop/Ingenieria-Informatica-2/AC/Practica 3 (+ S3)$ export OMP_SCHEDULE="guided,1"
./pmtv-OpenMP 8
Tiempo: 0.000269 segundos
Resultado:
res[0] = 8.00
res[1] = 7.00
res[2] = 6.00
res[3] = 5.00
res[4] = 4.00
res[5] = 3.00
res[6] = 2.00
res[7] = 1.00
david@DESKTOP-BQGQKB2:/mnt/c/Users/Davito/Desktop/Ingenieria-Informatica-2/AC/Practica 3 (+ S3)$

```

8. Contestar a las siguientes preguntas sobre el código del ejercicio anterior:

(a) ¿Qué número de operaciones de multiplicación y qué número de operaciones de suma realizan cada uno de los threads en la asignación static con monotonic y un chunk de 1?

RESPUESTA: Con static=1 las filas se reparten de forma circular Thread 0 -> Filas 0-3-6 y 15 multiplicaciones y 15 sumas y Thread 1 Filas->1-4-7 (12) y Thread 2 Filas-> 2 y 5 (9)

(b) Con la asignación dynamic y guided, ¿qué cree que debe ocurrir con el número de operaciones de multiplicación y suma que realizan cada uno de los threads?

RESPUESTA: Con dynamic y guided el reparto debe equilibrarse. Como las primeras filas tienen mas operaciones estas planificaciones se pueden repartir mejor que con static

(c) ¿Qué alternativa cree que debería ofrecer mejores prestaciones? Razonar la respuesta.

RESPUESTA: La mejor alternativa seria dynamic o guided obviamente porque compensan los desequilibrios entre filas. Guided para reducir la sobrecarga respecto a dynamic.

9. Obtener en atcgrid los tiempos de ejecución del código paralelo (usando, como siempre, -O2 al compilar) que multiplica una matriz triangular por un vector con las alternativas de planificación static, dynamic y guided para chunk de 1, 64 y el chunk por defecto para la alternativa (con **monotonic** en todos los casos). Usar un tamaño de vector N múltiplo del número de cores y de 64 que esté entre 11520 y 23040. El número de threads en las ejecuciones debe coincidir con el número de núcleos del computador. Rellenar la Tabla 3 dos veces con los tiempos obtenidos. Representar el tiempo para static, dynamic y guided en función del tamaño del chunk en una gráfica (representar los valores de las dos tablas). Incluir los scripts utilizado en el cuaderno de prácticas. **NOTA: Nunca ejecute en atcgrid código que imprima todos los componentes del resultado.**

CAPTURA CÓDIGO FUENTE: pmtv-OpenMP.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main(int argc, char **argv)
{
    int N, i, j;
    double **M, *v, *res;
    double t;

    if (argc < 2) {
        printf("Uso: %s <tamano_matriz>\n", argv[0]);
        return -1;
    }
}
```

```

}

N = atoi(argv[1]);

M = malloc(N * sizeof(double *));
v = malloc(N * sizeof(double));
res = malloc(N * sizeof(double));

for (i = 0; i < N; i++) {
    M[i] = malloc(N * sizeof(double));
}

for (i = 0; i < N; i++) {
    v[i] = 1.0;
    res[i] = 0.0;
    for (j = 0; j < N; j++) {
        if (j >= i)
            M[i][j] = 1.0;
        else
            M[i][j] = 0.0;
    }
}

t = omp_get_wtime();

#pragma omp parallel for private(j) schedule(runtime)
for (i = 0; i < N; i++) {
    for (j = i; j < N; j++) {
        res[i] += M[i][j] * v[j];
    }
}

t = omp_get_wtime() - t;

printf("Tiempo: %f segundos\n", t);

if (N <= 20) {
    printf("Resultado:\n");
    for (i = 0; i < N; i++) {
        printf("res[%d] = %.2f\n", i, res[i]);
    }
}

```

```
    for (i = 0; i < N; i++) {  
        free(M[i]);  
    }  
  
    free(M);  
    free(v);  
    free(res);  
  
    return 0;  
}
```

DESCOMPOSICIÓN DE DOMINIO: Se paraleliza el bucle exterior que recorre las filas de la matriz triangular superior, cada thread calcula distintas filas del vector segun la planificacion.

CAPTURAS DE PANTALLA:

```
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/
./pmtv-OpenMP_local.sh
N = 18432
Threads = 3
===== EJECUCION 1 =====
Static por defecto
Tiempo: 0.121387 segundos

Dynamic por defecto
Tiempo: 0.130798 segundos
Guided por defecto
Tiempo: 0.184983 segundos
Static chunk 1
Tiempo: 0.100771 segundos
Dynamic chunk 1
Tiempo: 0.102558 segundos
Guided chunk 1
Tiempo: 0.162429 segundos
Static chunk 64
Tiempo: 0.226221 segundos
Dynamic chunk 64
Tiempo: 0.097382 segundos
Guided chunk 64
Tiempo: 0.157841 segundos
===== EJECUCION 2 =====
Static por defecto
Tiempo: 0.125495 segundos
Dynamic por defecto
Tiempo: 0.092874 segundos
Guided por defecto
Tiempo: 0.126203 segundos
Static chunk 1
Tiempo: 0.098184 segundos
Dynamic chunk 1
Tiempo: 0.100742 segundos
Guided chunk 1
Tiempo: 0.127500 segundos
Static chunk 64
Tiempo: 0.089890 segundos
Dynamic chunk 64
Tiempo: 0.089157 segundos
Guided chunk 64
Tiempo: 0.135905 segundos
```

TABLA RESULTADOS, SCRIPT Y GRÁFICA atcgrid

SCRIPT: pmtv-OpenMP_atcgrid.sh

```
#!/bin/bash

N=18432
THREADS=3

export OMP_NUM_THREADS=$THREADS
```

```
gcc -O2 -fopenmp -o pmtv-OpenMP pmtv-OpenMP.c

echo "N = $N"
echo "Threads = $OMP_NUM_THREADS"

echo "==== EJECUCION 1 ====="

export OMP_SCHEDULE="monotonic:static"
echo "Static por defecto"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:dynamic"
echo "Dynamic por defecto"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:guided"
echo "Guided por defecto"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:static,1"
echo "Static chunk 1"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:dynamic,1"
echo "Dynamic chunk 1"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:guided,1"
echo "Guided chunk 1"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:static,64"
echo "Static chunk 64"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:dynamic,64"
echo "Dynamic chunk 64"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:guided,64"
echo "Guided chunk 64"
./pmtv-OpenMP $N
```

```
echo "==== EJECUCION 2 ====="

export OMP_SCHEDULE="monotonic:static"
echo "Static por defecto"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:dynamic"
echo "Dynamic por defecto"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:guided"
echo "Guided por defecto"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:static,1"
echo "Static chunk 1"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:dynamic,1"
echo "Dynamic chunk 1"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:guided,1"
echo "Guided chunk 1"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:static,64"
echo "Static chunk 64"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:dynamic,64"
echo "Dynamic chunk 64"
./pmtv-OpenMP $N

export OMP_SCHEDULE="monotonic:guided,64"
echo "Guided chunk 64"
./pmtv-OpenMP $N
```

Tabla 2 . Tiempos de ejecución de la versión paralela del producto de una matriz triangular por un vector **para vectores de tamaño N=18432**

Chunk	Static	Dynamic	Guided
por defecto	0.121387	0.130798	0.184983
1	0.100771	0.102558	0.162429
64	0.226221	0.097382	0.157841

Chunk	Static	Dynamic	Guided
por defecto	0.125495	0.092874	0.126203
1	0.098184	0.100742	0.127500
64	0.089890	0.089157	0.135905